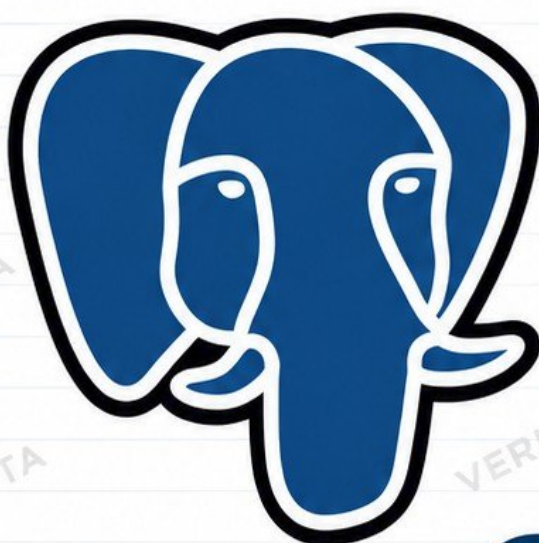




- ☐ Deploy
- ☐ Secure
- ☐ Monitor
- ☐ Troubleshoot
- ☐ Automate
- ☐ Scale

Learn
Build
Operate
Grow

Databases
Power
Modern
Applications



PostgreSQL

FOR DEVOPS ENGINEERS

From Database Fundamentals
to Production Operations

Reliable
Secure
Scalable



DEPLOY
& CONFIGURE



SECURE
& MANAGE



MONITOR
& OPTIMIZE



BACKUP
& RECOVER



AUTOMATE
& SCALE

PRACTICAL. REAL-WORLD. PRODUCTION-FOCUSED.

LINUX | DOCKER | KUBERNETES | CI/CD | MONITORING | BACKUPS | HIGH AVAILABILITY

Good
Databases
Build
Great Systems

ENGINEER TODAY
A MORE RELIABLE TOMORROW



VERIQT
KNOWLEDGE
SKILLS
OPPORTUNITIES
A BETTER YOU



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Data
Drives
Everything

PostgreSQL EXPLAINED,

WHY DEVOPS ENGINEERS NEED IT

From Database Fundamentals to Production Operations

Better
Infrastructure
Stronger
Applications

1 WHAT POSTGRESQL IS



- PostgreSQL is a powerful, open source, relational database management system (RDBMS).
- It stores, manages, and retrieves structured data.
- It is reliable, feature-rich, and widely used in production systems.
- Supports advanced features like replication, high availability, JSON, and more.

2 RELATIONAL DATABASE BASICS

- A relational database stores data in tables (rows and columns).
- Tables can be linked using relationships (e.g., foreign keys).
- This structure helps keep data organized, consistent, and easy to query.

Example: A simple users table

id	name	email
1	Alice	alice@example.com
2	Bob	bob@example.com
3	Carol	carol@example.com

3 DATABASE vs DATABASE SERVER

DATABASE



- A database is a collection of structured data (e.g., users, orders, products).
- You can have multiple databases on one server.

DATABASE SERVER



- A database server is the software (e.g., PostgreSQL) that runs and manages databases.
- It handles connections, security, queries, and stores data on disk.

4 WHY APPLICATIONS NEED DATABASES

- Applications need a database to store persistent data such as:
 - User accounts
 - Application settings
 - Transactions
 - Logs
 - Business data
- Without a database, data would be lost when the application stops.
- Databases allow multiple users and services to access the same data reliably and safely.



5 POSTGRESQL IN MODERN INFRASTRUCTURE

- Used in web applications, microservices, and cloud-native platforms.
- Commonly deployed on:
 - Linux servers
 - Docker containers
 - Kubernetes (StatefulSets)
 - Cloud managed services (e.g., AWS RDS, Azure Database for PostgreSQL, Google Cloud SQL)
- Integrates well with DevOps tools such as Terraform, Ansible, CI/CD pipelines, monitoring stacks (Prometheus, Grafana), and backup solutions.

6 DEVOPS vs DBA RESPONSIBILITIES

DEVOPS ENGINEER

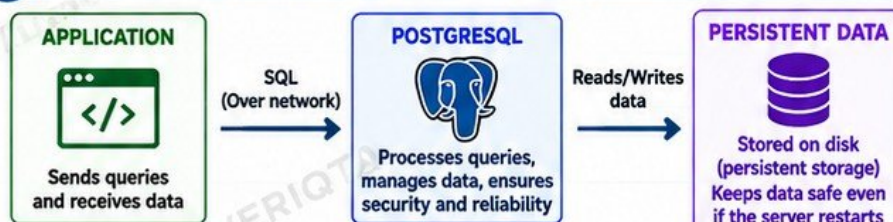
- ✓ Deploy and configure PostgreSQL
- ✓ Automate provisioning and backups
- ✓ Monitor availability and performance
- ✓ Manage infrastructure (Docker, Kubernetes, Cloud)
- ✓ Handle scaling and high availability
- ✓ Troubleshoot infrastructure and connectivity issues

DBA (DATABASE ADMIN)

- ✓ Database schema design
- ✓ Query optimization
- ✓ Index management
- ✓ Performance tuning
- ✓ Data modelling
- ✓ Advanced replication strategies
- ✓ Deep database internals
- ✓ Data migration strategies

In many teams, responsibilities overlap. DevOps engineers should understand database fundamentals to work effectively with DBAs.

7 MENTAL MODEL: APPLICATION → POSTGRESQL → PERSISTENT DATA



8 JUNIOR ENGINEER TIP



You do not need to be a DBA to operate databases responsibly.

As a DevOps engineer, understanding the basics helps you deploy, secure, monitor, and troubleshoot PostgreSQL in real environments.

SKILLS TODAY. STRONGER SYSTEMS TOMORROW.

PostgreSQL Architecture Without the Confusion

Understand the key components and how they work together

1 POSTGRESQL SERVER PROCESS



- The main process that manages database operations.
- Listens for client connections.
- Coordinates memory, disk I/O, background processes, and more.
- Runs as the postgres user.

2 CLIENT CONNECTIONS



- Clients connect using tools like psql or applications (e.g., web apps, microservices).
- Connect over TCP/IP (port 5432) or Unix sockets.
- Multiple clients can connect at the same time.

3 DATABASES



- A PostgreSQL server can have multiple databases.
- Each database is a separate container for schemas, tables, and other objects.
- Common system databases: postgres, template0, template1.

4 SCHEMAS



- A schema is a logical container inside a database.
- Helps organize tables, views, functions, and more.
- Default schema: public.
- You can create multiple schemas (e.g., dev, staging, prod).

5 TABLES



- Tables store data in rows and columns.
- Each table belongs to a schema.
- Tables have a defined structure (columns, data types, constraints).
- Indexes, triggers, and rules can be added for performance and automation.

6 ROLES



- Roles manage authentication and authorization.
- A role can be a user or a group.
- Roles can have different privileges (e.g., superuser, read-only).
- Use the principle of least privilege for security.

7 BACKEND PROCESSES



- Each client connection creates a backend process.
- Handles the client's queries.
- Multiple backend processes run at the same time.
- Other background processes handle tasks like checkpointing, WAL writing, and autovacuum.

8 SHARED MEMORY



- A memory area shared by all processes.
- Stores things like cached data (buffers), execution plans, locks, and more.
- Improves performance by reducing disk I/O.
- Key settings: shared_buffers, work_mem.

9 WAL (WRITE-AHEAD LOG)



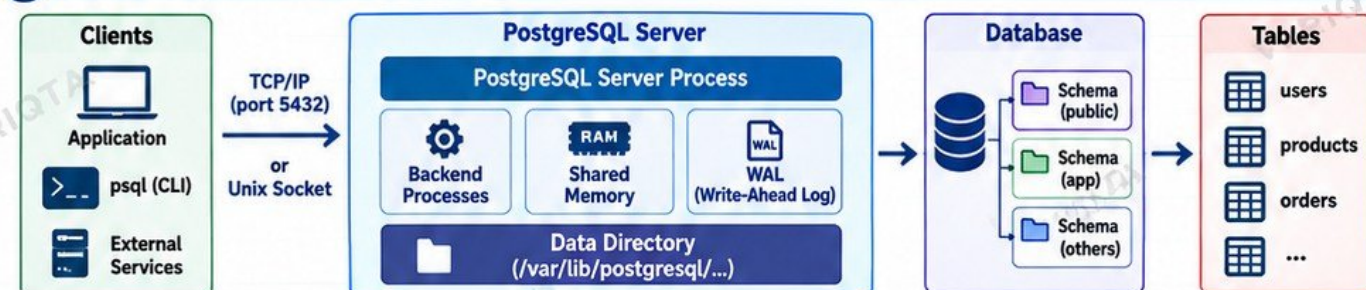
- Records all changes to the database before they are written to disk.
- Used for crash recovery, replication, and point-in-time recovery.
- Stored in the pg_wal directory (inside the data directory).
- Essential for data durability.

10 DATA DIRECTORY



- The main directory where PostgreSQL stores all data files, configuration files, WAL files, and more.
- Default location (Linux): /var/lib/postgresql/<version>/main
- Contains subdirectories for databases, tables, indexes, logs, and WAL.

11 POSTGRESQL ARCHITECTURE DIAGRAM



JUNIOR ENGINEER TIP



You do not need to be a DBA to operate PostgreSQL responsibly. But understanding its architecture helps you deploy, secure, monitor, and troubleshoot it effectively.



REMEMBER

A well-understood architecture leads to fewer outages and more reliable systems.

KNOWLEDGE BUILDS RELIABILITY

Installing PostgreSQL on Linux

Get PostgreSQL up and running on your Linux server

1 PACKAGE INSTALLATION



On Ubuntu/Debian:

```
sudo apt update
sudo apt install postgresql postgresql-contrib
```

On RHEL/CentOS/Rocky/AlmaLinux:

```
sudo dnf install postgresql postgresql-server
```

2 STARTING POSTGRESQL



Start the PostgreSQL service:

```
sudo systemctl start postgresql
```

Check status:

```
sudo systemctl status postgresql
```

3 CHECKING STATUS



View detailed service status:

```
sudo systemctl status postgresql
```

Expected output (running):

```
● postgresql.service - PostgreSQL RDBMS
   Loaded: loaded
   Active: active (running)
```

4 ENABLING STARTUP



Enable PostgreSQL to start on boot:

```
sudo systemctl enable postgresql
```



Created symlink
/etc/systemd/system/multi-user.target.wants/
postgresql.service → ...

5 POSTGRESQL SERVICE ACCOUNT



• PostgreSQL runs under its own Linux user (recommended for security).

• Check the user:

```
id postgres
```

• Switch to the postgres user:

```
sudo -i -u postgres
```

6 FINDING CONFIGURATION FILES



Main configuration file:

```
sudo find /etc -name postgresql.conf 2>/dev/null
```

Client authentication file:

```
sudo find /etc -name pg_hba.conf 2>/dev/null
```

On Ubuntu (common location):

- /etc/postgresql/<version>/main/postgresql.conf
- /etc/postgresql/<version>/main/pg_hba.conf

7 FINDING THE DATA DIRECTORY



Check data directory from psql:

```
sudo -u postgres psql -t -c "show data_directory;"
```

Or from config file:

```
grep data_directory /etc/postgresql  
/*main/postgresql.conf
```

Common location:

```
/var/lib/postgresql/<version>/main
```

8 CHECKING LISTENING PORTS



PostgreSQL default port: 5432

Check listening port:

```
sudo ss -tlnp | grep 5432
```

Or use netstat:

```
sudo netstat -tlnp | grep 5432
```

Expected output:

```
LISTEN 0      244      0.0.0.0:5432
LISTEN 0      244      [::]:5432
```

9 VERIFYING THE SERVER



Connect using psql:

```
sudo -u postgres psql
```

Check server version:

```
SELECT version();
```

List databases:

```
\l
```

Exit psql:

```
\q
```

10 PRACTICE LAB: INSTALL AND INSPECT POSTGRESQL

- 1 Install PostgreSQL on your Linux server.
- 2 Start the service and check its status.
- 3 Enable it to start on boot.
- 4 Identify the PostgreSQL service account.
- 5 Find the configuration files.
- 6 Find the data directory.
- 7 Check the listening port.
- 8 Connect using psql and verify the server version.
- 9 List the default databases.
- 10 Explore the directory structure inside the data directory.



You now have a working PostgreSQL installation and understand where key files and data are located.



JUNIOR ENGINEER TIP

You do not need to be a DBA to operate databases responsibly. Understanding how to install, run, and inspect PostgreSQL helps you diagnose issues, automate deployments, and work effectively with DBAs.



IMPORTANT

Do not run PostgreSQL as root. Always use the postgres user administration.

SAME COMMANDS. BIGGER OPPORTUNITIES.



LEARN
BUILD
SHARE
GROW

Connecting with psql

Access, Explore and Manage PostgreSQL from the Command Line

Same
Commands
Bigger
Opportunities

1 WHAT psql IS



- psql is the official command line client for PostgreSQL.
- It allows you to connect to a PostgreSQL server, run SQL commands, and manage databases.
- It is included with PostgreSQL and can be used on Linux, macOS, and Windows.



psql is a powerful tool for DevOps engineers to test, troubleshoot, and manage PostgreSQL.

2 LOCAL vs REMOTE CONNECTIONS

Local Connection



- Connects to PostgreSQL on the same server.
- Usually uses a Unix socket (default).
- No -h needed.

Remote Connection



- Connects to a PostgreSQL server on another server or VM.
- Uses TCP/IP (network).
- You must specify the host (-h), and usually the port (-p).

3 BASIC CONNECTION SYNTAX

```
psql -U <username> -h <host> -p <port> -d <database>
```

Option	Description
-U	Specify the username (role)
-h	Specify the host (IP address or hostname)
-p	Specify the port (default is 5432)
-d	Specify the database name

Example (remote connection):

```
psql -U appuser -h 192.168.1.10 -p 5432 -d mydb
```

4 COMMON EXAMPLES

Connect as default user (local):

```
psql
```

Connect as specific user:

```
psql -U postgres
```

Connect to a specific database:

```
psql -U appuser -d mydb
```

Connect to a remote server:

```
psql -U appuser -h 10.0.0.5 -p 5432 -d mydb
```

Use a connection string:

```
psql "postgresql://appuser:password@10.0.0.5:5432/mydb"
```

5 USEFUL psql COMMANDS

Command	What It Does
\l	List all databases
\c <dbname>	Connect to a database
\dt	List tables in current database
\d <table>	Show table structure
\du	List roles (users)
\q	Quit psql
\dn	List schemas
\dv	List views
\df	List functions
\x	Toggle expanded display
\?	Show help

6 EXAMPLE SESSION

```
$ psql -U postgres -d mydb
psql (15.5)
Type "help" for help.
```

```
mydb=# \l
```

Name	Owner	Encoding
mydb	postgres	UTF8
postgres	postgres	UTF8
template0	postgres	UTF8
template1	postgres	UTF8

(4 rows)

```
mydb=# \dt
```

```
No relations found.
```

```
mydb=# \q
```

```
$
```

7 CONNECTION FAILURES AND WHAT THEY MEAN

Error Message	Common Cause	How to Fix
could not connect to server	Server not running or wrong host/port	Check if PostgreSQL is running. Verify -h and -p.
connection refused	Port not open or PostgreSQL not listening	Check listen_addresses and firewall.
FATAL: password authentication failed	Wrong username or password	Check credentials and pg_hba.conf rules.
FATAL: database does not exist	Database name is incorrect	Check the database name with \l.
FATAL: role "user" does not exist	Username (role) is incorrect	Check \du or create the role.
no pg_hba.conf entry	Connection not allowed by pg_hba.conf	Add the correct rule and reload PostgreSQL.

8 JUNIOR ENGINEER TIP



You do not need to be a DBA to use psql effectively.

- Learn the basic commands.
- Understand how to connect locally and remotely.
- Use psql to verify, troubleshoot, and gather information.
- It is a simple but powerful tool every DevOps engineer should be comfortable with.



SECURITY REMINDER

Do not expose your PostgreSQL server to the internet. Use secure networks, firewalls, and strong authentication.

PRACTICE TODAY. STRONGER SYSTEMS TOMORROW.



LEARN
BUILD
SHARE
GROW

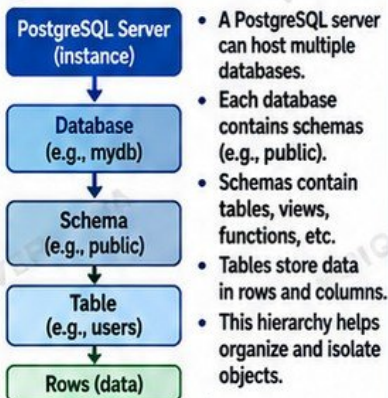
Databases, Schemas, Tables, and Data



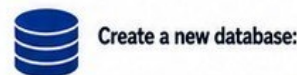
Same
Concepts
Bigger
Opportunities

Understand the structure. Work confidently with data.

1 DATABASE HIERARCHY



2 CREATING DATABASES



Create a new database:

```
CREATE DATABASE mydb;
```

List databases:

```
\l
```

Connect to a database:

```
\c mydb
```

3 CREATING SCHEMAS



Create a schema:

```
CREATE SCHEMA dev;
```

List schemas:

```
\dn
```

Set default schema (for current session):

```
SET search_path TO dev;
```

4 TABLES



Create a table:

```
CREATE TABLE dev.users (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100),
  created_at TIMESTAMP
  DEFAULT NOW()
);
```

5 COLUMNS



- Columns define the structure of the data in a table.
- Each column has a name and a data type.
- You can add constraints (e.g., NOT NULL, UNIQUE).

Example columns:

```
id SERIAL PRIMARY KEY,
name VARCHAR(100) NOT NULL,
age INTEGER,
email VARCHAR(100) UNIQUE,
created_at TIMESTAMP DEFAULT NOW()
```

6 PRIMARY KEYS



- A primary key uniquely identifies each row.
- It cannot be NULL.
- Common practice is to use SERIAL or BIGSERIAL for an auto-incrementing ID.
- Only one primary key per table.

Example:

```
id SERIAL PRIMARY KEY,
```

7 BASIC DATA TYPES



- INTEGER – whole numbers
- BIGINT – large numbers
- SERIAL – auto-increment integer
- VARCHAR(n) – variable length text
- TEXT – long text
- BOOLEAN – true or false
- DATE – date only
- TIMESTAMP – date and time
- NUMERIC(p, s) – exact numbers
- REAL / DOUBLE PRECISION – floating point numbers
- JSON / JSONB – JSON data
- UUID – unique identifier

8 BASIC SQL OPERATIONS

CREATE – create new data (table, etc.)

```
CREATE TABLE dev.products (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100)
);
```

INSERT – add data

```
INSERT INTO dev.products (name)
VALUES ('Laptop');
```

SELECT – retrieve data

```
SELECT * FROM dev.products;
```

UPDATE – modify data

```
UPDATE dev.products
SET name = 'MacBook' WHERE id = 1;
```

DELETE – remove data

```
DELETE FROM dev.products WHERE id = 1;
```

9 WHY DEVOPS ENGINEERS SHOULD UNDERSTAND BASIC SQL



- Helps you troubleshoot application issues (e.g., check data, verify records).
- Useful for automation, migrations, and CI/CD.
- Helps you work effectively with developers, DBAs, and data teams.
- Needed for monitoring, backups, and incident response.
- You don't need to be an expert, but basic SQL knowledge makes you more effective.



The goal is not to replace a DBA, but to understand enough to operate, troubleshoot, and automate confidently.

10 SAFE PRODUCTION AWARENESS



- Always be careful when running SQL in production.
- Use SELECT before UPDATE or DELETE to confirm data.
- Prefer running changes in a staging environment first.
- Use transactions for multiple changes:

```
BEGIN;
UPDATE dev.products SET name = 'New Name' WHERE id = 1;
-- check results
COMMIT; -- or ROLLBACK;
```

- Avoid running DROP or DELETE without a WHERE clause.
- Follow the principle of least privilege.
- Work with your DBA team for major schema changes.
- Always have backups before making significant changes.



A small SQL mistake in production can cause data loss. When in doubt, stop and ask.



JUNIOR ENGINEER TIP

You do not need to be a SQL expert to be a great DevOps engineer.

Understanding the basics helps you:

- Read and verify data
- Troubleshoot issues
- Automate tasks
- Communicate effectively
- Make safer changes

Learn the fundamentals, then practice regularly.

DATABASE KNOWLEDGE POWERS BETTER SYSTEMS

PostgreSQL Users, Roles, and Permissions



Control access. Follow least privilege. Keep your data safe.

1 ROLES vs USERS



- In PostgreSQL, everything is a role.
- A role can be a user (login role) or a group role (no login).
- Roles are used for authentication and authorization.
- There is no separate "user" object — users are roles with LOGIN.

Think of roles as identities that can have permissions and settings.

2 LOGIN ROLES



- A login role can connect to the database.
- Created with LOGIN attribute.
- Used by humans, applications, or services.
- Can have a password or use other authentication methods (e.g., certificate, IAM).

Create a login role:

```
CREATE ROLE appuser
WITH LOGIN PASSWORD 'StrongPass123!';
```

3 OWNERSHIP



- Every database object (database, schema, table, etc.) has an owner.
- The owner has full privileges on the object.
- You can change ownership using ALTER OWNER.

Change table owner:

```
ALTER TABLE dev.users
OWNER TO appuser;
```

4 CREATE ROLE



- Use **CREATE ROLE** to create new roles.
- You can specify LOGIN, PASSWORD, and other options.
- You can also create group roles (no login) and assign members.

Example:

```
CREATE ROLE readonly
WITH NOLOGIN;
CREATE ROLE appuser
WITH LOGIN PASSWORD 'Pass123!';
GRANT readonly TO appuser;
```

5 GRANT



- Use GRANT to give privileges to roles.
- You can grant privileges on databases, schemas, tables, columns, and more.
- You can also grant roles to other roles.
- Principle: grant only what is needed.

Example:

```
GRANT CONNECT ON DATABASE mydb
TO appuser;
```

6 REVOKE



- Use REVOKE to remove privileges.
- Important for security and cleanup.
- Always review and remove unnecessary privileges.

Example:

```
REVOKE INSERT ON dev.products
FROM appuser;
```

7 DATABASE PRIVILEGES



- Common database privileges:
- CONNECT** – allow connection
 - CREATE** – allow creating schemas
 - TEMPORARY** – allow temporary objects

Example:

```
GRANT CONNECT, CREATE ON
DATABASE mydb TO appuser;
```

8 SCHEMA PRIVILEGES



- Common schema privileges:
- USAGE** – allow access to objects in the schema
 - CREATE** – allow creating objects

Example:

```
GRANT USAGE, CREATE ON SCHEMA
dev TO appuser;
```

9 TABLE PRIVILEGES



- Common table privileges:
- SELECT** – read data
 - INSERT** – add data
 - UPDATE** – modify data
 - DELETE** – remove data
 - REFERENCES** – create foreign keys
 - TRIGGER** – create triggers

Example:

```
GRANT SELECT, INSERT, UPDATE
ON dev.products TO appuser;
```

10 SUPERUSER RISK



- Superusers have full access to everything.
- They can read, modify, or delete any data.
- Using superuser accounts in applications is a major security risk.
- Limit the number of superusers to trusted administrators only.



Do not use superuser accounts for applications.

11 LEAST PRIVILEGE



- Give only the permissions required to perform a task.
- Reduces security risk.
- Limits damage if credentials are compromised.
- Review permissions regularly.

- Start with minimum access
- Grant additional permissions only when needed
- Remove unused permissions
- Use separate roles for different applications

12 SERVICE ACCOUNTS FOR APPLICATIONS



- Create dedicated roles for applications.
- Grant only the required permissions.
- Avoid using personal accounts e.g., postgres or your own user.
- Use strong passwords or modern authentication methods (e.g., IAM, secrets management).
- Rotate credentials regularly.

13 WHY APPLICATIONS SHOULD NOT CONNECT AS POSTGRES



- The postgres role is a superuser by default.
- If compromised, an attacker gets full control of the database.
- Applications only need specific permissions, not unlimited access.
- Using dedicated, least-privilege roles improves security and auditability.
- Easier to track and manage access per application.



JUNIOR ENGINEER TIP

Always use a dedicated, least-privilege role for each application. Understand the difference between roles, privileges, and ownership. Good access control is a key part of running databases safely in production.

SECURE DATABASES. STRONGER SYSTEMS.

Understand
Configure
Troubleshoot
Operate

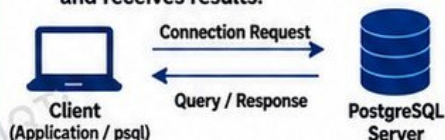
HOW POSTGRESQL CONNECTIONS ACTUALLY WORK

Same
Database
Stronger
Systems

Client, Network, Authentication and Everything in Between

1 CLIENT/SERVER CONNECTION MODEL

- A client (application, tool, or user) connects to PostgreSQL server.
- The client sends a connection request with authentication details.
- After successful authentication, a session is established.
- The client then sends SQL commands and receives results.



2 TCP/IP

- PostgreSQL uses TCP/IP for remote connections.
- Default port is 5432.
- Clients connect using the server's IP address or hostname.
- Works across networks (on-premises, cloud, or internet).

Example (remote connection)

```
psql -h 192.168.1.10 -p 5432 \
-U myuser -d mydb
```

3 UNIX SOCKETS

- On the same server, clients can connect using a Unix domain socket.
- Faster and more secure (no network traffic).
- Default socket directory (Linux): /var/run/postgresql/

Example (local connection)

```
psql -U myuser -d mydb
```

Uses the Unix socket by default if -h is not specified.

4 PORT 5432

- Default PostgreSQL port is 5432.
- You can change it in postgresql.conf:

```
port = 5432
```

- If you change the port, clients must use the new port.

5 listen_addresses

- Controls which network interfaces accept connections.
- Common values:
 - localhost (only local)
 - specific IP (e.g. 192.168.1.10)
 - * (all interfaces)

```
listen_addresses = 'localhost'
```



Do not use '*' in production unless properly secured.

6 DNS

- You can connect using a hostname instead of an IP.
- DNS helps clients find the database server.
- Useful in cloud environments and Kubernetes (using service names).

Example

```
psql -h db.example.com \
-p 5432 -U myuser -d mydb
```

7 CONNECTION STRINGS

- Applications use connection strings to connect.
- Common format (URI):

```
postgresql://user:password@
host:5432/dbname
```

Example

```
postgresql://appuser:secret@
db.example.com:5432/mydb
```

- Also supported:
 - Environment variables
 - Config files
 - Secrets (e.g., Kubernetes)

8 CONNECTION LIMITS

- PostgreSQL has a limit on the number of concurrent connections.
- Default is usually 100.
- Controlled by:

```
max_connections = 100
```

- Too many connections can cause performance issues or prevent new connections.



Use connection pooling (e.g., PgBouncer) for high traffic applications.

9 APPLICATION CONNECTION LIFECYCLE

- 1 Application starts
- 2 Resolves hostname (DNS)
- 3 Opens TCP connection (or Unix socket)
- 4 Sends authentication details
- 5 PostgreSQL verifies credentials (pg_hba.conf)
- 6 Connection established
- 7 Application sends queries
- 8 Connection stays open or is closed
- 9 On close, resources are released

10 COMMON CONNECTION ERRORS



connection refused

- PostgreSQL is not running.
- Wrong IP address or port.
- Server not listening (listen_addresses).
- Firewall or security group blocking port 5432.
- Network routing issue.



authentication failed

- Wrong username or password.
- User not allowed in pg_hba.conf.
- Incorrect authentication method.
- Connecting to the wrong message.
- Account is locked or does not exist.

11 NETWORK TROUBLESHOOTING FLOW



USEFUL COMMANDS



```
ss -ltnp | grep 5432
telnet <host> 5432
nc -zv <host> 5432
```

SHOW CONFIGURATION

```
SHOW listen_addresses;
SHOW port;
SHOW max_connections;
```

CHECK ACTIVE CONNECTIONS

```
SELECT count(*) FROM pg_stat_activity;
```



JUNIOR ENGINEER TIP

Most connection problems are either network, authentication, or configuration related. Read the exact error message.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

BUILD
SKILLS
CREATE
OPPORTUNITIES

Same
Database
Stronger
Systems

PostgreSQL

AUTHENTICATION AND pg_hba.conf

Secure Access, Correct Configuration, Reliable Systems

Security
Starts with
Correct
Access Control

1 WHAT pg_hba.conf CONTROLS

- pg_hba.conf controls how clients authenticate when connecting to PostgreSQL.
- It defines which users can connect, from where, to which databases, and how they must authenticate (e.g., password, SSL, cert, etc.).
- It is the first line of security for your database.



Configuration file location:
/var/lib/pgsql/data/pg_hba.conf
(or /etc/postgresql/<version>/main/pg_hba.conf)

2 HOST-BASED AUTHENTICATION

- Host-based authentication (HBA) checks the client's:
 - Connection type (local or host)
 - Database name
 - User name
 - Client IP address (for host)
 - Authentication method

Example rule:

```
host    all    all    192.168.1.0/24    scram-sha-256
```

Type Database User IP Range Auth Method

3 RULE ORDER

- Rules are checked from top to bottom.
- The first matching rule is used.
- More specific rules should be placed before general rules.
- If no rule matches, the connection is rejected.

**Order matters!**

A general rule at the top can override more specific rules below it.

4 LOCAL vs HOST RULES

**LOCAL**

- Used for connections via Unix domain sockets (on the same server).
- Example:

```
local all all peer
```

**HOST**

- Used for TCP/IP connections (from remote servers or clients).
- Example:

```
host all all 0.0.0.0/0 scram-sha-256
```

5 USER AND DATABASE MATCHING

- You can specify:
 - A specific database (e.g., mydb) or all
 - A specific user (e.g., appuser) or all
 - Match multiple users or databases using the same rule

Examples:

```
host mydb appuser 10.0.0.0/24 scram-sha-256
host all all 192.168.1.10 md5
host all all 0.0.0.0/0 reject
```

6 CIDR RANGES

- Use CIDR notation to allow specific IP ranges.
- Examples:

CIDR Range	What It Allows
192.168.1.0/24	All IPs from 192.168.1.1 to 192.168.1.254
10.0.0.0/16	All IPs from 10.0.0.1 to 10.0.255.254
0.0.0.0/0	All IPs (not recommended)
/32	A single IP address

7 MODERN PASSWORD AUTHENTICATION

- Recommended methods:
 - scram-sha-256 (most secure, default in modern PostgreSQL)
 - md5 (older, still used)
- Avoid trust, password (plain text), or ident in production.

Example:

```
host all all 192.168.1.0/24 scram-sha-256
```

8 TLS CONNECTIONS

- Use SSL/TLS to encrypt connections.
- Required files in postgresql.conf:
 - ssl = on
 - ssl_cert_file = 'server.crt'
 - ssl_key_file = 'server.key'
- Configure pg_hba.conf to require SSL:

```
hostssl all all 0.0.0.0/0 scram-sha-256
```

- Clients can connect using:

```
psql "host=db.example.com port=5432
sslmode=require user=appuser dbname=mydb"
```

9 RELOADING CONFIGURATION

- After editing pg_hba.conf:
- You do not need to restart PostgreSQL.
- Just reload the configuration:

```
sudo systemctl reload postgresql
```

- Or from psql:

```
SELECT pg_reload_conf();
```



Changes take effect immediately for new connections.

10 COMMON AUTHENTICATION MISTAKES

- ✗ Placing a general allow rule (e.g., 0.0.0.0/0) at the top.
- ✗ Using trust authentication in production.
- ✗ Incorrect order of rules.
- ✗ Wrong IP range or hostname.
- ✗ User or database name does not match.
- ✗ Forgetting to reload the configuration.
- ✗ Not enabling SSL for remote connections.
- ✗ Relying on password (plain text) instead of scram-sha-256.

11 SECURITY REMINDER



Never solve access problems by opening PostgreSQL to everyone.

- Do not use 0.0.0.0/0 in production.
- Use specific IP ranges and least privilege.
- Require SSL/TLS for remote connections.
- Use strong authentication methods (scram-sha-256).
- Regularly review and audit access rules.

**JUNIOR ENGINEER TIP**

If a connection fails, read the exact error message. It usually tells you whether the issue is network, authentication, or a mismatch in user/database.

Secure Databases
Enable Bigger Dreams



Right
Configuration
Stronger
Systems

PostgreSQL

CONFIGURATION FOR DEVOPS ENGINEERS

Understand. Configure. Optimize. Run with Confidence.

Same
Database
Bigger
Possibilities

1 postgresql.conf

- Main configuration file for PostgreSQL.
- Controls how the server behaves (memory, connections, logging, etc.).
- Typical location (Linux):

`/var/lib/pgsql/data/postgresql.conf`

or

`/etc/postgresql/<version>/main/postgresql.conf`



Edit with care. Always keep a backup before making changes.

4 ALTER SYSTEM

- Change configuration without editing the file manually.
- Settings are saved in `postgresql.auto.conf`.

```
ALTER SYSTEM SET max_connections = 200;
```

```
ALTER SYSTEM SET log_min_duration_statement = '1s';
```

```
ALTER SYSTEM RESET max_connections;
```



After using **ALTER SYSTEM**, reload the configuration for changes to apply.

2 CONFIGURATION HIERARCHY

- 1 Default compiled-in values (built into PostgreSQL)
- 2 `postgresql.conf` (main config file)
- 3 Included config files (e.g., `conf.d` directory)
- 4 **ALTER SYSTEM** (stored in `postgresql.auto.conf`)
- 5 Session level (SET command, temporary)

The last setting in the hierarchy overrides the previous ones.

3 SHOW

- View the current value of a parameter.
- Useful for checking settings and troubleshooting.

```
SHOW max_connections;
SHOW shared_buffers;
SHOW log_min_duration_statement;
SHOW all; -- shows all parameters
```



Use **SHOW** to verify changes take effect.

5 RELOAD vs RESTART

RELOAD

- ✓ Applies most configuration changes.
- ✓ Does not drop connections.
- ✓ Faster and safer (used in production).
- ✓ Command:

```
sudo systemctl reload postgresql
```

OR

```
SELECT pg_reload_conf();
```

RESTART

- ✓ Required for some settings (e.g., changes to shared memory).
- ✓ Drops existing connections.
- ✓ Short downtime.
- ✓ Command:

```
sudo systemctl restart postgresql
```

Use restart only when required.

6 max_connections

- Maximum number of concurrent client connections.
- Default is usually 100.
- Setting it too high can cause memory issues.

```
max_connections = 200
```



Consider using a connection pooler (e.g., PgBouncer) instead of increasing this too much.

7 MEMORY SETTINGS

- Important memory parameters:

```
shared_buffers = 25% of RAM
work_mem = memory per sort/operation
maintenance_work_mem = for maintenance
effective_cache_size = OS cache estimate
```



Do not copy memory settings from other companies. Tune based on your environment and workload.

8 LOGGING SETTINGS

- Helps with monitoring and troubleshooting.
- Key settings:

```
log_destination = 'stderr'
logging_collector = on
log_directory = 'log'
log_filename = 'postgresql-%Y-%m-%d.log'
log_min_duration_statement = 1000 # ms
log_connections = on
log_disconnections = on
```



Logs are essential for debugging performance and security issues.

9 TIMEOUTS

- Prevent hanging connections and long-running queries.

```
statement_timeout = 30000 # 30s
idle_in_transaction_session_timeout = 60000
lock_timeout = 10000
tcp_keepalives_idle = 600
tcp_keepalives_interval = 60
tcp_keepalives_count = 5
```



Set sensible timeouts to keep your database healthy.

10 WAL SETTINGS

- WAL (Write-Ahead Log) ensures durability and enables replication and point-in-time recovery.
- Key settings:

```
wal_level = replica
max_wal_size = 1GB
min_wal_size = 80MB
checkpoint_timeout = 5min
checkpoint_completion_target = 0.9
```

11 CONFIGURATION CHANGES IN PRODUCTION

- ✓ Understand the impact of each change.
- ✓ Test in a non-production environment first.
- ✓ Make small, incremental changes.
- ✓ Monitor after the change (logs, metrics, performance).
- ✓ Have a rollback plan.
- ✓ Use automation (Ansible, Terraform, etc.) for consistency.
- ✓ Document all changes.

12 WHY COPYING TUNING VALUES IS DANGEROUS

- ✗ Every environment is different (hardware, workload, usage patterns).
- ✗ Values from another company may not fit your infrastructure.
- ✗ Can cause poor performance, instability, or even outages.
- ✗ Always tune based on real metrics and testing.



JUNIOR ENGINEER TIP:

Understand what each setting does, test changes, monitor the impact, and tune for your environment. Good configuration is a skill, not a copy and paste.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LEARN
SHARE
BUILD
GROW

Reliable
Data
Resilient
Systems

Data
Today
Available
Tomorrow

PostgreSQL

STORAGE, WAL, AND DATABASE DURABILITY

How PostgreSQL Stores, Protects, and Keeps Your Data Safe

1 WHERE POSTGRESQL STORES DATA

- PostgreSQL stores all its data on the server's filesystem.
- Data includes databases, tables, indexes, configuration files, WAL logs, and more.
- Everything is stored in the data directory (PGDATA).



Stored on disk
(not in memory)

2 DATA DIRECTORY (PGDATA)

- The data directory contains all database files.
- Default locations:


```
/var/lib/pgsql/data (RHEL/CentOS)
/var/lib/postgresql/<version>/main (Ubuntu)
$PGDATA (custom)
```
- Contains subdirectories for databases, WAL, configuration, etc.
- Check the location:


```
SHOW data_directory;
```

3 PAGES

- Data is stored in fixed-size pages (usually 8 KB).
- Each table, index, and other object is made up of pages.
- Pages are read into memory (shared buffers) when needed.
- Changes are first written to WAL, then to data pages on disk.



Default page size:
8 KB
(can be changed at initdb)

4 WAL (WRITE-AHEAD LOG)

- WAL is a sequential log of all changes made to the database.
- Every change is written to WAL first, before being written to the actual data files.
- Located in `$PGDATA/pg_wal`



Example:

```
/var/lib/postgresql/15/main/pg_wal/
```

5 WHY WAL EXISTS

- Provides durability and crash recovery.
- If the server crashes, PostgreSQL uses WAL to recover recent changes.
- Enables features like point-in-time recovery and replication.
- Ensures data is not lost, even if the system goes down unexpectedly.



WAL = Safety net for your data

6 CHECKPOINTS

- A checkpoint writes dirty pages (from memory) to disk.
- Helps keep the database consistent and reduces WAL size.
- Controlled by settings like:

```
checkpoint_timeout = 5min
max_wal_size = 1GB
```



Checkpoints do not replace WAL. They work together.

7 fsync

- fsync ensures data is physically written to disk, not just to OS cache.
- By default, fsync = on (recommended).
- Improves durability but may slightly affect performance.

```
fsync = on
```



Do not disable fsync in production unless you fully understand the risks. It can lead to data loss.

8 PERSISTENT STORAGE

- Use persistent storage (e.g., EBS, local SSD, SAN, or persistent volumes in Kubernetes).
- Avoid ephemeral storage for production databases.
- Ensure the storage is reliable, has good IOPS, and is backed up.



Your data is only as reliable as your storage.

9 DISK CAPACITY

- Monitor disk usage regularly.
- PostgreSQL needs space for:
 - Data files
 - WAL files
 - Temporary files
 - Logs
 - Autovacuum and maintenance
- If disk is full, PostgreSQL can stop accepting writes and may become unusable.



Set up monitoring and alerts for disk space.

10 DISK LATENCY

- Low disk latency is important for good performance.
- Use SSDs or high-performance storage (e.g., gp3, io2 on AWS).
- High latency can cause slow queries, lag, and timeouts.



Faster storage = better performance and reliability.

11 FILESYSTEM CONSIDERATIONS

- Use a stable, production-grade filesystem (ext4, XFS are common).
- Avoid filesystems that are not tested with PostgreSQL.
- Mount with recommended options (e.g., noatime).
- Ensure proper filesystem tuning and monitoring.



Filesystem stability matters for database reliability.

12 WHAT HAPPENS WHEN STORAGE FILLS

- PostgreSQL may stop accepting writes.
- Transactions can fail.
- WAL cannot be written, leading to a database shutdown.
- Your application will experience errors and downtime.



Monitor disk usage and set up alerts before you run out of space.

13 WHY DELETING RANDOM FILES FROM PGDATA IS DANGEROUS

- PostgreSQL files are tightly linked and must stay consistent.
- Deleting random files can corrupt the database.
- You may lose data permanently.
- Always use proper tools (e.g., VACUUM, pg_basebackup, WAL management) to manage disk space.



Never manually delete files from PGDATA unless instructed in official documentation.



JUNIOR ENGINEER TIP:

Understand how PostgreSQL stores and protects data. Good storage planning and monitoring prevents outages and data loss.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LEARN
SHARE
BUILD
GROW



PostgreSQL BACKUPS

Protect Your Data. Ensure Business Continuity.

Back Up
Recover
Stay Ready

Reliable Backups for Real-World PostgreSQL Environments

1 BACKUP vs HIGH AVAILABILITY

BACKUP	HIGH AVAILABILITY
- A copy of your data. Used for recovery after accidental delete, corruption, or disaster. - Can be stored offline or off-site. - Point-in-time recovery possible.	- Keeps your database running if a server fails (e.g., replication, failover). - Reduces downtime but is NOT a backup. - You still need backups even with high availability.

Use both: High availability for uptime, backups for data protection.

2 LOGICAL vs PHYSICAL BACKUPS

LOGICAL BACKUP	PHYSICAL BACKUP
- Backs up database objects (schemas, tables, data). - Uses pg_dump, pg_dumpall. - Output is SQL (human readable). - Good for migrations and selective restores.	- Backs up the entire data directory (binary files). - Uses base backups (e.g., pg_basebackup). - Faster for large databases. - Required for point-in-time recovery with WAL.

3 pg_dump (SINGLE DATABASE)

Creates a backup of one database.

```
pg_dump -U postgres -d myapp \
-F c -f myapp.dump
```

- F c = custom format (recommended)
- Other formats: -F p (plain), -F t (tar), -F d (directory)
- You can also compress on the fly:

```
pg_dump -U postgres -d myapp | \
gzip > myapp.sql.gz
```

4 pg_dumpall (ALL DATABASES)

Creates a backup of all databases and roles.

```
pg_dumpall -U postgres -f all_databases.sql
```

- Includes global objects like roles and tablespaces.

5 pg_restore (RESTORE A BACKUP)

Restores a database from a custom format backup.

```
pg_restore -U postgres -d myapp \
myapp.dump
```

- Use -l to list contents: pg_restore -l myapp.dump
- Use -v for verbose output.

6 BASE BACKUPS (PHYSICAL)

Creates a physical base backup of the entire cluster.

```
pg_basebackup -U postgres -D /backup/pgbase \
-Fp -Xs -P
```

- F p = plain format (directory)
- X s = include WAL for point-in-time recovery
- Can be used with streaming replication.

7 BACKUP FORMATS

Format	Option	Description
Plain SQL	-F p	Human readable SQL file
Custom	-F c	Compressed, flexible (recommended)
Tar	-F t	Tar archive
Directory	-F d	Directory format (large DBs)

8 COMPRESSION

- Reduces backup size and storage cost.
- Use gzip, bzip2, or xz.

```
pg_dump -U postgres -d myapp | \
gzip > myapp.sql.gz
```

- For large databases, use parallel compression tools (e.g., pigz).

9 SCHEDULING BACKUPS

- Use cron for regular backups.
- Automate and monitor.

```
# Example: daily backup at 2 AM
0 2 * * * /usr/local/bin/pg_backup.sh
```

- Log output and check for failures.
- Consider using tools like pgBackRest or Barman for advanced scheduling and management.

10 RETENTION

- Keep multiple backup copies (e.g., daily, weekly, monthly).
- Define retention period based on business needs.
- Example: keep daily for 7 days, weekly for 4 weeks, monthly for 6 months.

11 OFF-SERVER STORAGE

- Store backups outside the production server.
- Options:
 - Cloud storage (AWS S3, Azure Blob, Google Cloud Storage)
 - Remote server via SCP/rsync
 - Backup tools (pgBackRest, Barman)
 - Object storage with lifecycle policies

12 ENCRYPTION

- Protect backups at rest and in transit.
- Use GPG for encryption:


```
pg_dump -U postgres -d myapp | \
gzip | gpg -c > myapp.sql.gz.gpg
```
- Use secure key management.
- Ensure encrypted backups are accessible when needed.

13 BACKUP VERIFICATION

- Regularly test your backups.
- Perform test restores in a non-production environment.
- Check data integrity.
- Automate verification if possible.
- A backup you have never restored is not enough.



JUNIOR ENGINEER TIP

A backup you have never restored is not enough.

Always test your backups. The real value of a backup is in a successful restore.

Backup Today
Peace Tomorrow



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Plan
Test
Recover
Repeat

RESTORE AND DISASTER RECOVERY

Be Ready Today. Recover Faster Tomorrow.



No Backup
No Business
Test Your
Recovery

1 WHY RESTORE TESTING MATTERS

- A backup is only useful if you can successfully restore it.
- Restores verify that your backups are valid and complete.
- Helps you identify missing files, permission issues, or configuration problems.
- Builds confidence during real incidents.
- Should be tested regularly, not just during emergencies.



"If you have never restored your backup, you do not have a backup."

2 RESTORING SQL DUMPS

Use psql for plain SQL dumps or pg_restore for custom format.

Plain SQL dump

```
psql -U postgres -d myapp -f \
  backup.sql
```

- Recreates the database objects and data from the SQL file.
- Works with files created using pg_dump (plain format).

Create database first (if needed)

```
createdb -U postgres myapp
```

3 pg_restore (CUSTOM FORMAT)

Use pg_restore for custom or compressed backups.

```
pg_restore -U postgres -d myapp \
  -v backup.dump
```

- Used for custom format backups (created with pg_dump -Fc).
- Supports parallel restore (-j).
- Provides detailed output (-v).
- Allows selective restore of schemas, tables, or objects.

Tip: Always restore into a test environment first before restoring to production.

4 RECOVERY OBJECTIVES

Define how much data you can afford to lose and how quickly you need to recover.

RPO (Recovery Point Objective)



The maximum acceptable amount of data loss (e.g., 5 minutes, 1 hour, 1 day).

RTO (Recovery Time Objective)



The maximum acceptable time to restore service (e.g., 1 hour, 4 hours, 1 day).

5 POINT-IN-TIME RECOVERY (PITR)

Allows you to restore your database to a specific point in time using WAL (Write-Ahead Logs).

- Recovers data up to a specific timestamp or LSN (Log Sequence Number).
- Requires a base backup and archived WAL files.
- Useful for accidental data deletion or corruption.

Key files

- Base backup (full data)
- Archived WAL files
- recovery.conf / recovery.signal (configuration for recovery)

6 WAL ARCHIVING CONCEPT

WAL (Write-Ahead Log) records all changes made to the database.

- WAL files are archived to a safe, off-server location.
- During recovery, PostgreSQL replays WAL files to reach the desired point in time.
- Common archiving destinations:
 - Local directory
 - Remote server (SCP/rsync)
 - Cloud storage (AWS S3, Azure Blob, Google Cloud Storage)
- Configure using archive_mode and archive_command in postgresql.conf.

7 RECOVERY VERIFICATION

- ✓ Confirm the database starts successfully.
- ✓ Verify all expected data is present.
- ✓ Check application connectivity.
- ✓ Validate key tables and row counts.
- ✓ Test application functionality.
- ✓ Review logs for errors.
- ✓ Document the results.

8 DISASTER RECOVERY RUNBOOKS



- Include clear, step-by-step instructions.
- Define roles and responsibilities.
- List required tools and access.
- Document RPO and RTO targets.
- Include communication plan.
- Test the runbook regularly.
- Keep it updated after infrastructure.

In a real incident, you do not want to figure it out. You want a runbook.

9 PRACTICE LAB

Backup → Destroy Test Database → Restore

- 1 Create a test database and sample data.
- 2 Take a backup (pg_dump).
- 3 Drop the test database.
- 4 Restore from the backup.
- 5 Verify the data and application connectivity.

10 RECOVERY FLOW



JUNIOR ENGINEER TIP

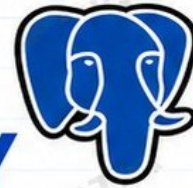
A backup you have never restored is not enough.

Make restore testing part of your regular work. Real recovery builds real confidence.

PREPARE PRACTICE RECOVER STAY READY

Reliable
Databases
Stronger
Systems

PostgreSQL REPLICATION AND HIGH AVAILABILITY



Scale
Replicate
Failover
Stay Online

Keep Your Database Online. Always.

1 WHY REPLICATION EXISTS



- Provides high availability and data durability.
- Keeps a copy of your data on another server.
- Reduces downtime during failures, maintenance, or upgrades.
- Allows read scaling for heavy read workloads.
- Helps meet business continuity and disaster recovery goals.

Replication ensures your database stays available even when the primary server is not.

2 PRIMARY AND STANDBY

- **Primary:** accepts all write operations (INSERT, UPDATE, DELETE).
- **Standby (Replica):** maintains a read-only copy of the primary.
- Standby continuously receives WAL records from the primary and replays them.
- Multiple standbys can be used for read scaling and redundancy.



3 STREAMING REPLICATION

- Real-time replication using WAL records.
- Primary sends WAL to standby over a network connection.
- Standby replays WAL to keep in sync.
- Low latency and efficient.
- Commonly used in production environments.

```
# On primary (check status)
SELECT * FROM
pg_stat_replication;
```

4 WAL SHIPPING

- WAL files are archived and copied to a standby server (e.g., via rsync or cloud storage).
- Standby retrieves and replays the archived WAL files.
- Usually used for disaster recovery or when real-time streaming is not possible.



5 REPLICATION LAG



- The time difference between data on the primary and standby.
- Caused by network latency, high load, or slow standby.
- Check lag using:

```
SELECT now() - pg_last_xact_
replay_timestamp() AS lag;
```

- High replication lag can lead to data loss during failover.

6 SYNCHRONOUS vs ASYNCHRONOUS

SYNCHRONOUS REPLICATION

- Primary waits for standby to confirm receipt of WAL.
- Stronger data consistency (no or minimal data loss).
- Higher latency.
- Used when data loss is not acceptable.

ASYNCHRONOUS REPLICATION

- Primary does not wait for standby confirmation.
- Lower latency and higher performance.
- Small risk of data loss during failover.
- Common in most production environments.

Choose synchronous for critical data, asynchronous for better performance.

7 FAILOVER

- Promote the standby to become the new primary when the primary fails.
- Can be manual or automated (e.g., using Patroni, repmgr, or cloud managed services).
- Ensure applications reconnect to the new primary.



8 SWITCHOVER

- Planned failover.
- Used for maintenance or controlled transitions.
- Promote standby to primary and optionally make the old primary a new standby.
- Helps avoid downtime.

9 SPLIT-BRAIN CONCEPT

- Occurs when both primary and standby accept writes at the same time.
- Leads to data inconsistency.
- Prevent using proper fencing, quorum, and cluster management tools (e.g., etcd, Patroni, or cloud native solutions).



10 REPLICATION IS NOT A BACKUP

- Replication protects against server failure, not human error, accidental deletion, or corruption.
- Standby usually has the same data, including mistakes.
- You still need regular backups (e.g., pg_dump, base backups) and test restores.



11 HA ARCHITECTURE



JUNIOR ENGINEER TIP

Replication gives you high availability, but backups give you recovery. You need both.

Same Data
More Uptime
Bigger Opportunities

Monitor
Detect
Troubleshoot
Stay Ahead

Monitoring PostgreSQL IN PRODUCTION

Visibility Today. A More Reliable Tomorrow.



Metrics
Logs
Alerts
Reliable Databases

1 WHAT SHOULD BE MONITORED?

- ✓ Availability (is it up and responding?)
- ✓ Connections (current and max)
- ✓ Query latency (slow queries)
- ✓ Transactions (commits, rollbacks)
- ✓ Locks (blocking and waiting)
- ✓ Cache behavior (hit/miss ratios)
- ✓ Disk usage (data, WAL, temp)
- ✓ WAL generation (write volume)
- ✓ Replication lag (standby delay)
- ✓ Database size (growth trends)

Monitoring gives you early warnings before small issues become big incidents.

2 POSTGRESQL IN A MONITORING STACK



Collect → Store → Visualize → Alert → Take Action

"You cannot improve what you cannot see."
— Operating PostgreSQL without monitoring is a risk.

3 AVAILABILITY

- Is PostgreSQL running?
- Can it accept connections?
- Is it responding to queries?
- Monitor uptime and response time.
- Use simple checks (e.g., `pg_isready`).

```
pg_isready -h localhost -p 5432 -U postgres
```

- ✓ Alert if the database is down or not responding.

4 KEY METRICS TO MONITOR

Area	Important Metrics
Connections	Active connections, max_connections
Query Latency	Slow queries, query duration
Transactions	Commits, rollbacks, transactions/sec
Locks	Blocked queries, lock wait time
Cache Behavior	Cache hit ratio, shared_buffers usage
Disk Usage	Database size, table/index bloat
WAL Generation	WAL size, write rate
Replication Lag	Replay lag, replication state
Database Size	Total size, growth rate
Temp Files	Temporary file usage (sorts, hashes)

5 USEFUL POSTGRESQL STATISTICS VIEWS

Common views for monitoring:

View	What it Shows
<code>pg_stat_activity</code>	Active sessions and queries
<code>pg_stat_database</code>	Database-level statistics
<code>pg_stat_user_tables</code>	Table access and size stats
<code>pg_stat_user_indexes</code>	Index usage statistics
<code>pg_stat_replication</code>	Replication status and lag
<code>pg_stat_bgwriter</code>	Background writer activity
<code>pg_stat_wal</code>	WAL generation statistics
<code>pg_locks</code>	Lock information
<code>pg_stat_statements</code>	Query performance (if enabled)

```
SELECT * FROM pg_stat_activity;
```

6 PROMETHEUS POSTGRESQL EXPORTERS

- Use the `postgres_exporter` to collect PostgreSQL metrics.
- Supports standard Prometheus metrics.
- Easy to deploy (Docker, Kubernetes, or bare metal).
- Collects key metrics like connections, transactions, replication lag, cache stats, WAL, etc.

```
docker run -d \
  --name postgres-exporter \
  -p 9187:9187 \
  prometheuscommunity/postgres-exporter
```

The exporter uses a read-only user. Do not use the postgres superuser.

7 GRAFANA DASHBOARDS

- Use pre-built PostgreSQL dashboards (e.g., from Grafana Labs or community).
- Visualize key metrics: connections, query latency, disk usage, replication lag, etc.
- Create custom dashboards for your environment.
- Helps you spot trends and issues quickly.



8 ALERTING ON SYMPTOMS THAT MATTER

Alert Condition	Why It Matters
Database down	Service unavailable
High connection count	Risk of hitting max_connections
Long query duration	Performance issue
High number of blocked queries	Applications may be stuck
Replication lag above threshold	Standby is not in sync
Disk space low	Can cause database outage
High WAL generation	May indicate unusual activity
Repeated deadlocks	Application issues
High temp file usage	Inefficient queries
Unusual increase in database size	Storage planning needed

Alert on symptoms, not just metrics.
Focus on what impacts availability, performance, or data safety.

9 PRODUCTION MONITORING TIPS

- ✓ Monitor the right metrics, not everything.
- ✓ Set meaningful thresholds.
- ✓ Use multiple channels for alerts (Email, Slack, PagerDuty).
- ✓ Track trends over time.
- ✓ Include replication and disk monitoring.
- ✓ Regularly review and tune alerts.
- ✓ Combine metrics with logs for better context.
- ✓ Test your alerting (don't wait for an incident).
- ✓ Document your monitoring setup.

10 JUNIOR ENGINEER TIP

Monitoring is not just about seeing problems. It helps you build a more reliable, stable, and scalable PostgreSQL environment.

MONITOR TODAY. PREVENT TOMORROW.

PostgreSQL

SLOW QUERIES, LOCKS, AND PERFORMANCE TROUBLESHOOTING

Find the Bottleneck. Fix the Problem. Keep Your Database Fast.



Observe
Analyze
Troubleshoot
Optimize

Find
Fix
Optimize
Keep It Fast

1 WHAT MAKES A QUERY SLOW?

- Missing indexes
- Scanning large tables
- Complex joins
- Poorly written queries
- Returning too much data
- High concurrent load
- Insufficient hardware (CPU, memory, I/O)
- Locks and blocking
- Outdated table statistics
- Inefficient query plans



A slow query is usually a symptom of a deeper issue. Always investigate the root cause.

2 ACTIVE QUERIES

See what queries are currently running in the database.

```
SELECT pid, username, datname, state,
       query, query_start
FROM pg_stat_activity
WHERE state != 'idle'
ORDER BY query_start;
```

- Shows real-time queries.
- Helps identify long-running queries.
- Useful during performance issues.

3 pg_stat_activity

The main view for monitoring active sessions and queries.

```
SELECT * FROM
pg_stat_activity;
```

- Shows session state (active, idle, idle in transaction, etc.).
- Shows query text, runtime, client address.
- Helps find blocking queries and long-running transactions.

4 LOCKS

Locks control access to data and prevent conflicts.

- Row-level locks (most common)
- Table-level locks
- Locks can block other queries and cause slow performance.
- Check current locks:

```
SELECT * FROM pg_locks;
```

Locks are normal, but long-running locks can bring your application to a halt.

5 BLOCKING SESSIONS

Find which sessions are blocking others.

```
SELECT blocked.pid AS blocked_pid,
       blocker.pid AS blocking_pid,
       blocked.query AS blocked_query,
       blocker.query AS blocking_query
FROM pg_stat_activity blocked
JOIN pg_locks b1 ON blocked.pid = b1.pid
JOIN pg_locks b2 ON b1.locktype = b2.locktype
AND b1.database IS NOT DISTINCT FROM b2.database
AND b1.relation IS NOT DISTINCT FROM b2.relation
AND b1.page IS NOT DISTINCT FROM b2.page
AND b1.tuple IS NOT DISTINCT FROM b2.tuple
JOIN pg_stat_activity blocker ON b2.pid = blocker.pid
WHERE NOT b1.granted AND b2.granted;
```

- Identifies which query is blocking another.
- Helps you decide which session to terminate.

6 LONG-RUNNING TRANSACTIONS

- Transactions that stay open for a long time can:
 - Hold locks
 - Prevent vacuum from cleaning data
 - Cause table and index bloat
 - Increase replication lag

Find long-running transactions:

```
SELECT pid, username, datname,
       state, xact_start, query
FROM pg_stat_activity
WHERE state = 'active'
AND now() - xact_start > interval '5 minutes'
ORDER BY xact_start;
```

Keep transactions short. Avoid idle transactions in application code.

7 INDEX BASICS

- Indexes help PostgreSQL find data faster.
- Without indexes, PostgreSQL may perform a full table scan.
- Use indexes on:
 - Primary keys
 - Foreign keys
 - Columns used in WHERE, JOIN, and ORDER BY
 - Frequently queried columns
- Check existing indexes:


```
\di
```
- Create an index:


```
CREATE INDEX idx_name
ON table_name (column_name);
```



Indexes improve read performance but also add storage and write overhead.

8 EXPLAIN AND EXPLAIN ANALYZE

Use EXPLAIN to see the query plan.

```
EXPLAIN
SELECT * FROM users
WHERE email = 'test@example.com';
```

- Shows how PostgreSQL plans to execute the query.
- Use EXPLAIN ANALYZE to see actual execution time.

```
EXPLAIN ANALYZE
SELECT * FROM users
WHERE email = 'test@example.com';
```

- Shows real execution time, rows scanned, and bottlenecks.
- Compare estimated vs actual rows.

EXPLAIN ANALYZE is your best friend for query performance tuning.

9 CPU, MEMORY, STORAGE, OR QUERY PROBLEM?



- CPU**
- High CPU usage
 - Many active queries
 - Complex queries



- MEMORY**
- High memory usage
 - Large sorts/joins
 - Insufficient shared_buffers



- STORAGE**
- High disk I/O
 - Slow disk latency
 - Full disk or low IOPS

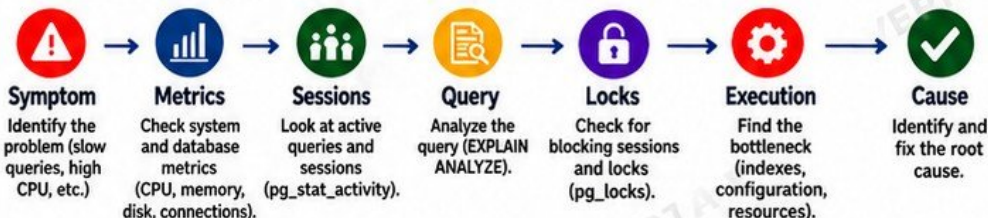


- QUERY**
- Missing indexes
 - Inefficient query plan
 - Long-running queries

Always check all layers: query, system resources, locks, and configuration.

10 TROUBLESHOOTING WORKFLOW

Follow a step-by-step approach to find and fix the root cause.



Systematic troubleshooting saves time and prevents recurring issues.

11 JUNIOR ENGINEER TIP



Slow queries are not just a database problem. They are often an application, index, or infrastructure problem. Stay curious, investigate, and always verify the root cause.

OBSERVE INVESTIGATE SOLVE IMPROVE

Better Queries
Happier Users
Stronger Systems



VACUUM, AUTOVACUUM, AND DATABASE MAINTENANCE

Keep Your PostgreSQL Databases Healthy, Fast, and Reliable

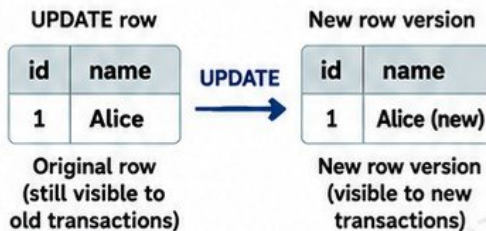
Maintain
Optimize
Keep Running

1 POSTGRESQL MVCC CONCEPT

- PostgreSQL uses Multi-Version Concurrency Control (MVCC).
- When data is modified, PostgreSQL creates a new row version instead of overwriting the old one.
- The old version is kept so ongoing transactions can still see a consistent view of the data.
- Each row has hidden system columns like xmin and xmax to track visibility.

MVCC allows high concurrency but also creates dead tuples.

2 WHY UPDATES CREATE DEAD TUPLES



The old row becomes a dead tuple and is not immediately removed. VACUUM is needed to clean it up.

3 VACUUM

- VACUUM removes dead tuples and reclaims space for reuse.
- Does not reduce the physical file size (use VACUUM FULL if you need to shrink the file).
- Keeps the database performant.
- Can be run manually:

```
VACUUM;
```

- For aggressive cleanup:

```
VACUUM (VERBOSE, ANALYZE);
```

4 AUTOVACUUM

- A background process that automatically runs VACUUM and ANALYZE.
- Monitors tables and runs based on thresholds (e.g., number of changed rows).
- Essential for preventing table bloat and transaction ID wraparound.
- Configured in `postgresql.conf` (e.g., `autovacuum = on`).
- Do NOT disable autovacuum.

Autovacuum keeps your database healthy without manual intervention.

5 ANALYZE

- ANALYZE collects statistics about the data in tables.
- Helps the query planner choose the best execution plan.
- Usually run automatically by autovacuum.

- Can be run manually:

```
ANALYZE;
```

- Or for a specific table:

```
ANALYZE my_table;
```

6 TABLE STATISTICS

- PostgreSQL stores statistics about table data in system catalogs.
- Used by the query planner for better performance.
- Useful views:

```
SELECT * FROM pg_stat_user_tables;
```

- Check table size and dead tuples:

```
SELECT relname, n_live_tup,
n_dead_tup
FROM pg_stat_user_tables;
```

7 TABLE AND INDEX BLOAT

- Bloat happens when dead tuples are not cleaned up.
- Tables and indexes grow larger than necessary.
- Leads to:
 - Higher disk usage
 - Slower queries
 - More I/O
 - Increased backup size
- Monitor bloat using tools like `pgstattuple` or `pg_stat_user_tables`.

Regular vacuuming keeps bloat under control.

8 TRANSACTION ID WRAPAROUND

- Every transaction gets a unique transaction ID (XID).
- XIDs are 32-bit integers and will eventually wrap around.
- If old transactions are not cleaned up (by vacuum), the database can stop accepting writes.
- Autovacuum prevents this by cleaning old tuples before XID wraparound becomes a problem.
- Check age:

```
SELECT datname, age(datfrozenxid)
FROM pg_database;
```

9 WHY DISABLING AUTOVACUUM IS DANGEROUS

- ✗ Dead tuples accumulate.
- ✗ Tables and indexes bloat.
- ✗ Query performance degrades.
- ✗ Disk space usage increases.
- ✗ Transaction ID wraparound risk.
- ✗ Can lead to database downtime.
- ✗ Manual vacuuming is not reliable at scale.

Never disable autovacuum in production.

10 MAINTENANCE MONITORING

- ✓ Monitor autovacuum activity:

```
SELECT * FROM
pg_stat_activity
WHERE query LIKE '%vacuum%';
```

- ✓ Track dead tuples:

```
SELECT relname, n_dead_tup
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC;
```

- ✓ Monitor table and index sizes.

- ✓ Alert on high bloat.

- ✓ Monitor XID age.

- ✓ Include in your monitoring stack (e.g., Prometheus + Grafana).

11 PRODUCTION MAINTENANCE MINDSET

- ✓ Let autovacuum run and tune it if needed.
- ✓ Monitor, don't guess.
- ✓ Watch for bloat, long-running transactions, and XID age.
- ✓ Schedule maintenance during low-traffic periods for heavy tasks (e.g., VACUUM FULL, REINDEX).
- ✓ Treat database maintenance as part of your DevOps responsibility.
- ✓ Keep your database healthy for reliability, performance, and lower costs.



JUNIOR ENGINEER TIP

A backup you have never restored is not enough.

The same applies to maintenance. Monitor, test, and verify that your database stays healthy.

Healthy Databases
Reliable Systems



PostgreSQL WITH DOCKER AND KUBERNETES

Deploy. Persist. Scale. Operate. The Right Way.

Containers
Orchestrate
Reliable
Databases

1 RUNNING POSTGRESQL IN A CONTAINER

- Use the official PostgreSQL image Docker Hub.
- Quick way to run a test or development database.
- Not suitable for production without persistent storage.



```
docker run --name pg \
-e POSTGRES_PASSWORD=strongpassword \
-p 5432:5432 \
postgres:16
```

2 CONTAINER FILESYSTEM vs PERSISTENT DATA

- Container filesystem is ephemeral (data is lost when container is deleted).
- Databases need persistent storage.
- Always use volumes for database data.

Container
filesystem
(ephemeral)

❌
Data lost
on container removal

Volume
(persistent)

✅
Data survives
container restarts

3 DOCKER VOLUMES

- Docker volumes persist data outside the container.
- Recommended over bind mounts.

```
docker volume create pgdata
docker run -d --name pg \
-v pgdata:/var/lib/postgresql/data \
-e POSTGRES_PASSWORD=strongpassword
postgres:16
```

4 KUBERNETES STATEFULSETS

- StatefulSets are designed for stateful applications like databases.
- Provides stable network identity, ordered deployment, and persistent storage.
- Each Pod gets a stable name (e.g., postgres-0).



```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres
  replicas: 1
  selector:
    matchLabels:
      app: postgres
```

5 PERSISTENT VOLUMES (PV)

- PV is storage in the cluster (e.g., EBS, GCE PD, NFS).
- Managed by the cluster administrator.



```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: postgres-pv
spec:
  capacity:
    storage: 20Gi
  accessModes:
    - ReadWriteOnce
  storageClassName: gp3
```

6 PERSISTENT VOLUME CLAIMS (PVC)

- PVC requests storage from a PV.
- Pods use PVCs to get persistent storage.



```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 20Gi
  storageClassName: gp3
```

7 SERVICES

- Provides a stable endpoint to access PostgreSQL.
- Use a ClusterIP for internal access.
- Use a headless Service for StatefulSets.



```
apiVersion: v1
kind: Service
metadata:
  name: postgres
spec:
  clusterIP: None # headless
  selector:
    app: postgres
  ports:
    - port: 5432
```

8 CONFIGMAPS

- Store non-sensitive configuration (e.g., postgresql.conf).
- Mount as a file or environment variable.



```
apiVersion: v1
kind: ConfigMap
metadata:
  name: postgres-config
data:
  postgresql.conf: |
    max_connections = 200
    shared_buffers = 256MB
```

9 SECRET REFERENCES

- Store sensitive data (e.g., passwords).
- Use Kubernetes Secrets and reference them in Pods.



```
env:
  - name: POSTGRES_PASSWORD
    valueFrom:
      secretKeyRef:
        name: postgres-secret
        key: password
```

10 PROBES

- Use liveness and readiness probes to keep the database healthy.
- Helps with automatic recovery and traffic management.



```
livenessProbe:
  exec:
    command: ["pg_isready", "-U", "postgres"]
  initialDelaySeconds: 10
  periodSeconds: 10
readinessProbe:
  exec:
    command: ["pg_isready", "-U", "postgres"]
  initialDelaySeconds: 5
  periodSeconds: 10
```

11 RESOURCE REQUESTS AND LIMITS

- Set CPU and memory requests and limits.
- Prevents noisy neighbors and improves stability.



```
resources:
  requests:
    cpu: "250m"
    memory: "512Mi"
  limits:
    cpu: "1"
    memory: "1Gi"
```

12 STATEFUL WORKLOAD RISKS



- Data loss if storage is not configured correctly.
- Pod rescheduling issues.
- Split brain in multi-replica setups.
- Disk space exhaustion.
- Network and DNS misconfiguration.
- Accidental deletion of PVCs.
- Upgrades without proper backups.

13 WHY DEPLOYMENT IS USUALLY WRONG FOR A PRODUCTION DATABASE



- Deployments are designed for stateless applications.
- Pods can be replaced with new identities.
- Does not provide stable storage, ordering, or stable network identity.
- Risk of data loss or corruption.
- Use StatefulSets instead for databases.

14 MANAGED vs SELF-MANAGED POSTGRESQL

Feature	Managed PostgreSQL (e.g., AWS RDS, Azure, GCP)	Self-Managed on Kubernetes
Setup	Fast and simple	More complex
Operations	Handled by cloud provider	Your responsibility
Backups	Automated	You must configure
Scaling	Easy (limited control)	Full control
Maintenance	Automated (patching, upgrades)	You manage
Cost	Pay for convenience	Lower cost but higher effort
Use Case	Most production workloads	Advanced use cases / full control



JUNIOR ENGINEER TIP

A database in a container is only as reliable as its storage.

Always use persistent storage, test your backups, and understand how your platform handles stateful workloads.

Same Skills
A More Reliable
Tomorrow



VERIQT
KNOWLEDGE
SKILLS
OPPORTUNITIES
A BETTER YOU



PostgreSQL AUTOMATION, CI/CD, AND INFRASTRUCTURE AS CODE

Automate Today. Reliable Databases Tomorrow.

Code
Automate
Deliver
Repeat

1 INFRASTRUCTURE PROVISIONING

- Provision PostgreSQL using Infrastructure as Code (IaC).
- Use cloud providers (AWS, Azure, GCP) or self-managed Kubernetes.
- Define compute, storage, networking, security, and backup from the start.



```
# Example: Terraform (AWS RDS)
resource "aws_db_instance" "postgres" {
  engine = "postgres"
  instance_class = "db.t3.medium"
  allocated_storage = 100
  db_name = "myapp"
}
```

2 TERRAFORM AND MANAGED POSTGRESQL

- Use Terraform to provision managed PostgreSQL (RDS, Azure DB for PostgreSQL, Cloud SQL).
- Benefits: automated, repeatable, version controlled.



Terraform

```
module "postgres" {
  source = "terraform-aws-modules/rds/aws"
  engine = "postgres"
  engine_version = "15"
  instance_class = "db.t3.medium"
  allocated_storage = 100
}
```

3 CONFIGURATION AUTOMATION

- Automate PostgreSQL configuration using Ansible, Terraform, or Kubernetes manifests.
- Manage postgresql.conf, pg_hba.conf, and extensions.



Ansible



Terraform



Kubernetes

```
# Ansible example
- name: Configure postgresql.conf
  lineinfile:
    path: /var/lib/pgsql/data/postgresql.conf
    regexp: '^max_connections'
    line: 'max_connections = 200'
```

4 SECRETS MANAGEMENT

- Never hardcode database credentials.
- Use a secrets manager:
 - AWS Secrets Manager
 - Azure Key Vault
 - Google Secret Manager
 - Kubernetes Secrets



5 DATABASE MIGRATIONS

- Use migration tools (Flyway, Liquibase, or application framework).
- Keep migrations version controlled.
- Test migrations in non-production environments.



Flyway



Liquibase

```
# Flyway example
flyway -url=jdbc:postgresql://db:5432/myapp \
-user=app -password=***** migrate
```

6 CI/CD MIGRATION STAGES

- Build application
- Run tests
- Run database migrations
- Deploy application
- Verify health

7 MIGRATION ORDERING

- Apply dependent changes in the correct order.
- Use separate migration files with clear numbering.
- Avoid breaking application compatibility.

```
V1_create_users.sql
V2_add_email_column.sql
V3_create_orders.sql
V4_add_index.sql
```

8 BACKWARD-COMPATIBLE SCHEMA CHANGES

- Use additive changes first (e.g., add columns, not remove).
- Keep odd columns until the application is fully migrated.
- Use feature flags when needed.

GOOD EXAMPLE

- ✓ Add new column (nullable)
- ✓ Deploy application (write to both)
- ✓ Backfill data
- ✓ Switch reads
- ✓ Remove old column later

9 ROLLBACK CHALLENGES

- Database changes are harder to roll back than application code.
- Some changes (e.g., dropping columns) are irreversible.
- Plan rollbacks before running migrations.

ROLLOBACK STRATEGIES

- ✓ Use backward-compatible changes
- ✓ Keep backups before major changes
- ✓ Use transactional migrations
- ✓ Have a documented rollback plan

10 NEVER HARDCODE DATABASE CREDENTIALS

- Do NOT store credentials in code, Docker images, or CI/CD pipelines.
- Use environment variables or secrets management.
- Rotate credentials regularly.

NEVER DO THIS
DB_USER=postgres
DB_PASSWORD=admin123

DO THIS INSTEAD

- ✓ Use a secrets manager
- ✓ Use environment variables
- ✓ Use Kubernetes Secrets
- ✓ Rotate credentials

11 DEVELOPMENT → STAGING → PRODUCTION



- Feature development
- Test migrations
- Use non-sensitive data



- Production-like setup
- Run full migration tests
- Validate monitoring and backups



- Controlled deployments
- Monitor closely
- Have rollback plan

12 GITOPS CONSIDERATIONS

- Store infrastructure and database configuration in Git.
- Use pull requests and code reviews.
- Deploy changes through GitOps tools (e.g., Argo CD, Flux).



Argo CD



Flux

13 AUTOMATED BACKUP AND MONITORING CONFIGURATION

- Automate backups (e.g., AWS RDS snapshots, pg_dump, or WAL archiving).
- Configure monitoring (e.g., Prometheus, Grafana).
- Alert on key metrics: connections, replication lag, disk usage, backup failures.



Backups



Grafana



Prometheus



JUNIOR ENGINEER TIP

Automation reduces human error, but understanding what is being automated is what makes you reliable. Use automation, but always know how to do the tasks manually.

Automate
Secure
Scale
Succeed



@veriqta



veriqta



veriqta



veriqta



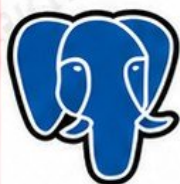
@veriqta



@veriqta



VERIQTA
KNOWLEDGE
SKILLS
OPPORTUNITIES
A BETTER YOU



FINAL PRODUCTION LAB

DEPLOY, BREAK, DIAGNOSE, AND RECOVER POSTGRESQL

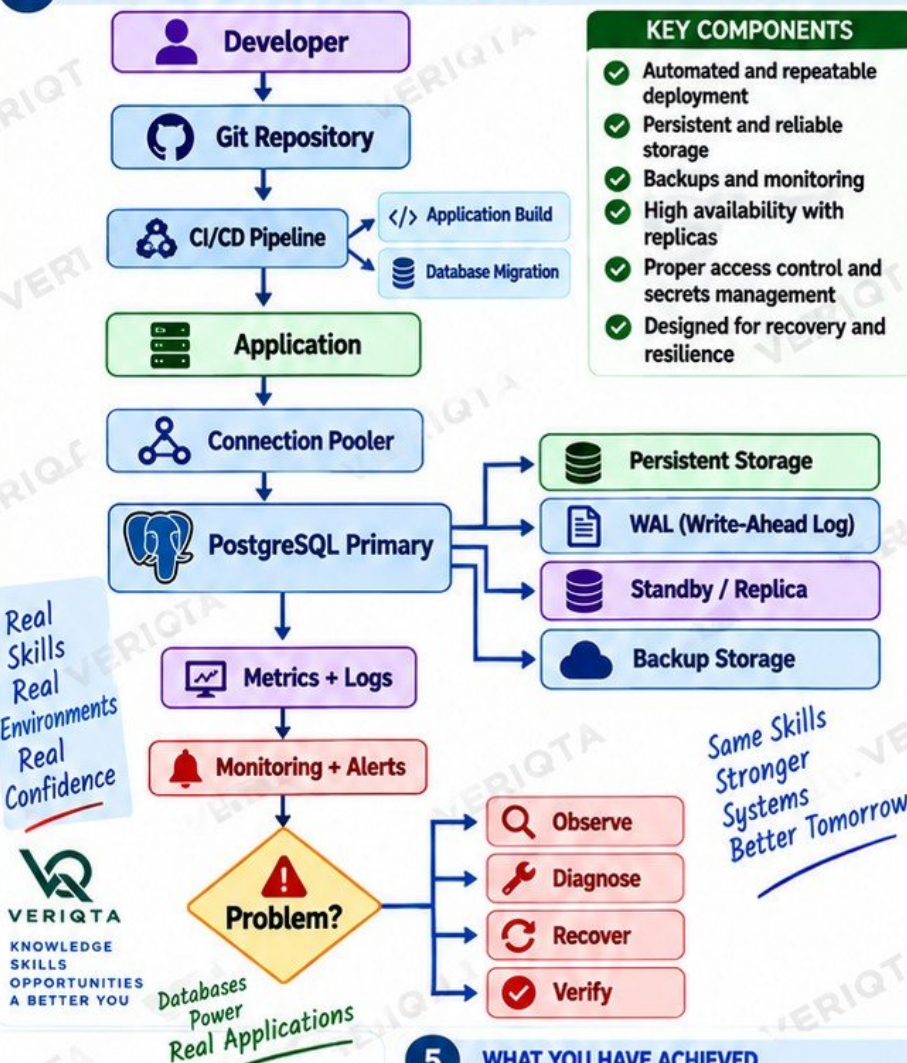
Turn Knowledge into Real-World Skills

Build
Operate
Troubleshoot
Recover

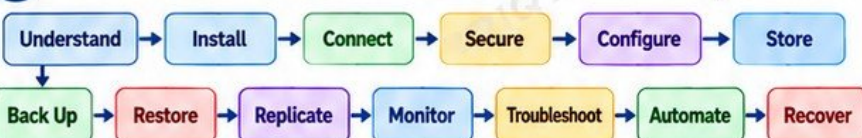
1 LAB STEPS

- 1 Provision PostgreSQL
- 2 Create application database
- 3 Create least-privilege application role
- 4 Configure authentication
- 5 Connect an application
- 6 Configure persistent storage
- 7 Enable useful logging
- 8 Configure backup
- 9 Add monitoring
- 10 Generate test data
- 11 Simulate a bad connection
- 12 Diagnose authentication/network failure
- 13 Simulate a slow or blocked query
- 14 Inspect sessions and locks
- 15 Take a backup
- 16 Simulate data loss in the lab
- 17 Restore the database
- 18 Verify application connectivity
- 19 Verify data integrity
- 20 Document the recovery procedure

2 FINAL PRODUCTION ARCHITECTURE



3 LEARNING PROGRESSION



4 JUNIOR ENGINEER TIP



A database is not just a service to install, it is a critical part of your system. Practice deploying, breaking, and recovering it. Real confidence comes from doing.

5 WHAT YOU HAVE ACHIEVED

- Deployed a production-style PostgreSQL setup
- Configured security, storage, logging, backup and monitoring
- Diagnosed and fixed real-world issues
- Practiced recovery from data loss
- Built the skills to operate PostgreSQL with confidence

5 WHAT YOU HAVE ACHIEVED

Practice Today
Production Tomorrow

Practicing
Production Tomorrow
Today



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta